The Problem: Traditional CRUD Operations

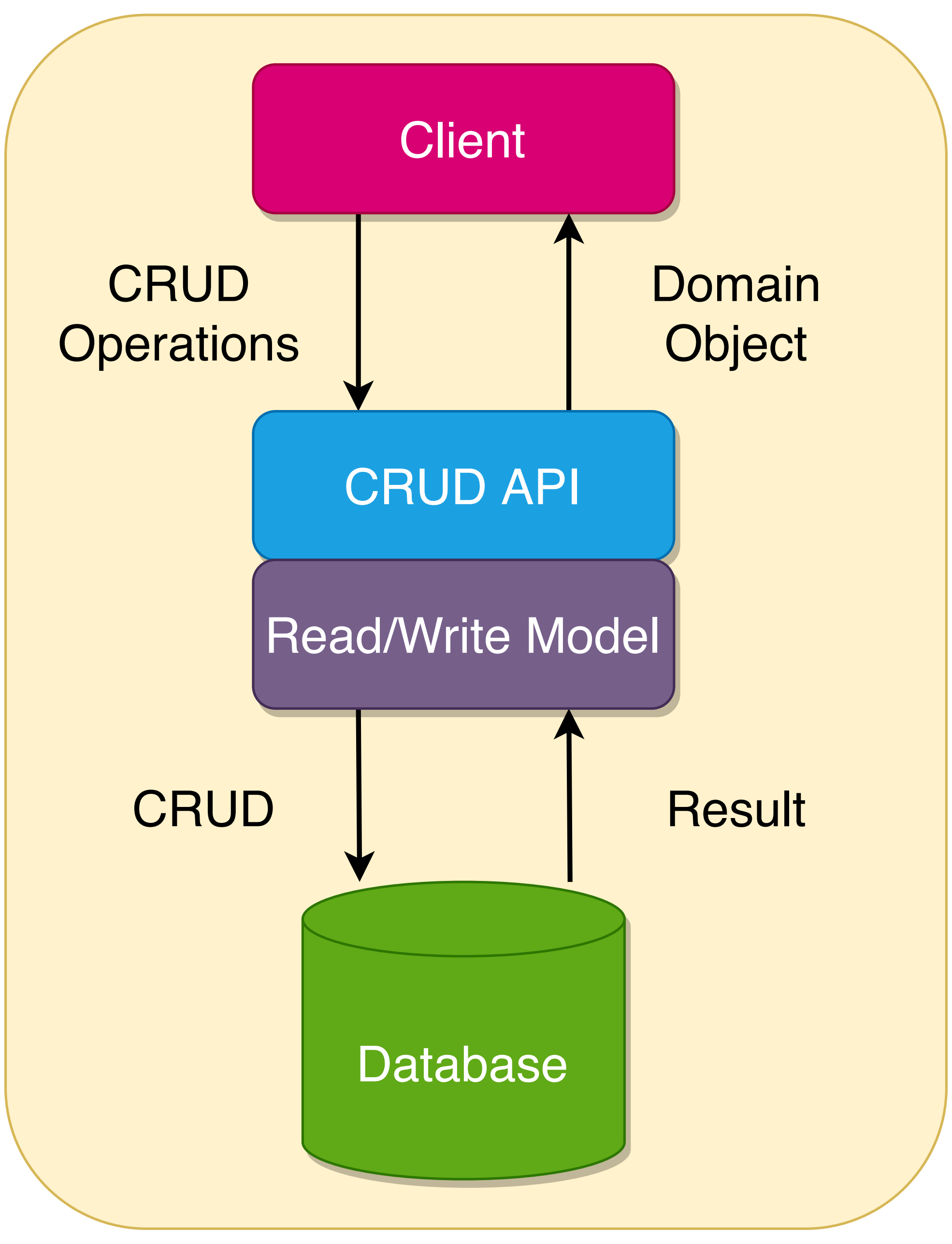
Limitations of CRUD

In the ideal world of application development, CRUD (create, read, update, delete) operations provide a simple and intuitive method for handling data. However, complex applications often demand handling numerous concurrent operations, diverse data requirements, and need to scale efficiently. Here, traditional CRUD operations sometimes struggle to keep up.

Consider a large e-commerce platform with a broad user base. The read and write operations for such a system differ drastically. You have thousands of users browsing products (read operations), while concurrently, you have inventory updates, order placements, and product additions (write operations). Can you visualize the pressure on the system? Can a single model efficiently handle these varied data requirements? That's where the cracks begin to show.

The Problem of Scale and Performance

One significant issue with traditional CRUD operations is scaling. As we've seen with our e-commerce example, there's a considerable difference in the scale of read and write operations. While read operations are significantly more, write operations carry more complexity. Can we scale them independently based on their needs? Unfortunately, with a single data model handling both operations, scaling becomes a tough challenge.



Typical monolith application

Let's understand this with an example.

Imagine you're working on a banking application. This application needs to handle various tasks like processing transactions (like deposits and withdrawals) and keeping track of account balances. Here's where things get tricky:

- **Multiple Operations**: In a banking system, you have two main types of operations:
  - **Transactions**: These are actions like depositing or withdrawing money.
  - **Balance Inquiries**: Users often want to check their current account balance.
- **The Challenge**: The problem arises when you try to handle these operations efficiently. Why? Because the needs of reading data (like checking balances) and writing data (like processing transactions) are quite different.
  - **Writing Data (Transactions)**: This involves creating new records or updating existing ones. It needs to be accurate and secure, as it directly affects a user's finances.
  - **Reading Data (Balance Inquiries)**: This is about fetching data quickly and efficiently. Users expect to see their balance in real-time, without any delay.
- **Complexity in Balancing Both**: Trying to optimize a system for both reading and writing data can be challenging. If you focus too much on making transactions fast and secure, you might end up making balance inquiries slower. On the other hand, if you optimize for quick balance checks, you might compromise the efficiency or security of processing transactions.

This scenario is a classic example of where a traditional approach to managing data falls short. It becomes difficult to maintain, scale, and ensure the system performs well under different types of loads. That's where CQRS comes into play, offering a solution to this dilemma by separating the concerns of reading and writing data.

← Previous
Introduction
Next →
CQRS Pattern: A Solution
☐ Mark as Completed